
Security Review Report
NM-0641 - Vishwa Morpho Blue P2



NETHERMIND
SECURITY

(March 02, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Architecture & Core Components	4
4.1.1	TieredLiquidationMorpho	4
4.1.2	WhitelistRegistry	5
4.1.3	HealthFactorLib	5
4.2	Liquidation Flows	5
4.2.1	One-Step Liquidation (liquidate)	5
4.2.2	Two-Step Liquidation	5
5	Risk Rating Methodology	6
6	Issues	7
6.1	[Critical] The TieredLiquidationMorpho extension can be bypassed through direct calls to the core Morpho contract	7
6.2	[High] Inconsistent liquidation incentive factor causes reverts and incorrect fee calculations	8
6.3	[High] Health factor check in the TieredLiquidationMorpho contract uses stale market data	9
6.4	[Medium] Reentrancy in _enforceNoActiveLock allows cooldown bypass and excess liquidation	10
6.5	[Low] Pending liquidations can be executed in disabled markets	10
6.6	[Low] Residual allowance in _executeMorphoLiquidation prevents subsequent liquidations for non-standard tokens	11
6.7	[Low] Unauthorized liquidations possible during market initialization	12
6.8	[Low] Whitelisted addresses cannot be restricted from performing one-step liquidations	12
6.9	[Low] lockDuration changes affect pending liquidation requests	13
6.10	[Info] Inconsistent expiry boundary handling across liquidation request lifecycle	13
6.11	[Info] The TieredLiquidationMorpho contract does not implement IMorphoLiquidateCallback	14
6.12	[Info] claimFailedRefund permanently locks ETH for contracts that cannot receive it	14
7	Documentation Evaluation	15
8	Test Suite Evaluation	16
8.1	Automated Tools	16
8.1.1	AuditAgent	16
9	About Nethermind	17

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for [Vishwa](#) smart contracts. Vishwa implements a specialized extension layer built upon a fork of Morpho Blue. This extension introduces a tiered liquidation framework that enables both restricted and public access controls. Specifically, the system provides two distinct mechanisms for managing undercollateralized positions: a one-step liquidation path (accessible to the public or whitelisted users) and a two-step liquidation process exclusive to whitelisted liquidators.

The audit comprises 510 lines of the Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools. **Along this document, we report** 12 points of attention, where one is classified as Critical, two are classified as High, one is classified as Medium and five are classified as Low, three are classified as Informational and Best Practices. The issues are summarized in Fig. 1.

Security Findings Exceed Expected Levels

Based on the size of the codebase (510 lines of code) and our historical audit metrics, we would typically expect an average of one issue per 150 lines of code, corresponding to approximately 3 to 4 issues for a project of this scope. However, the audit identified 12 issues, equivalent to approximately one issue every 42 lines of code, which is significantly above our baseline expectations.

Furthermore, our historical data shows that roughly 10% of reported issues fall into the high or critical severity categories. For a codebase of this size, this would normally translate to zero or one high/critical issues. Instead, this audit uncovered three high/critical severity issues, exceeding expected levels of risk.

These findings indicate that the overall security posture of the codebase is below what would normally be anticipated for a project of this size and complexity and warrants particular attention from the development team.

Security Failures and Inadequate Testing

During the engagement, it was noted that the provided documentation lacked comprehensive details regarding the precise intended behavior and integration mechanics of the new extension. While these architectural intentions were later clarified during sync calls with the developers, the initial ambiguity impacted the assumptions the Nethermind Security team used to evaluate the codebase. Additionally, the provided test suite is currently non-functional.

The security review uncovered severe architectural flaws, most notably a Critical vulnerability that allows the extension's core safeguards to be bypassed entirely via direct interactions with the Morpho singleton. Furthermore, the Nethermind Security team identified multiple systemic issues in which the extension's internal calculations diverge from Morpho's dynamic formulas, leading to blocked liquidations and potential bad-debt accumulation.

Given the number of high-severity issues relative to the codebase size, the nature of these integration mismatches and the absence of a functional test environment, it is strongly advised to repair and expand the test suite to ensure comprehensive code coverage. Once the highlighted issues are resolved and the protocol is thoroughly tested, it is highly recommended that the project undergo a subsequent security review prior to any mainnet deployment.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

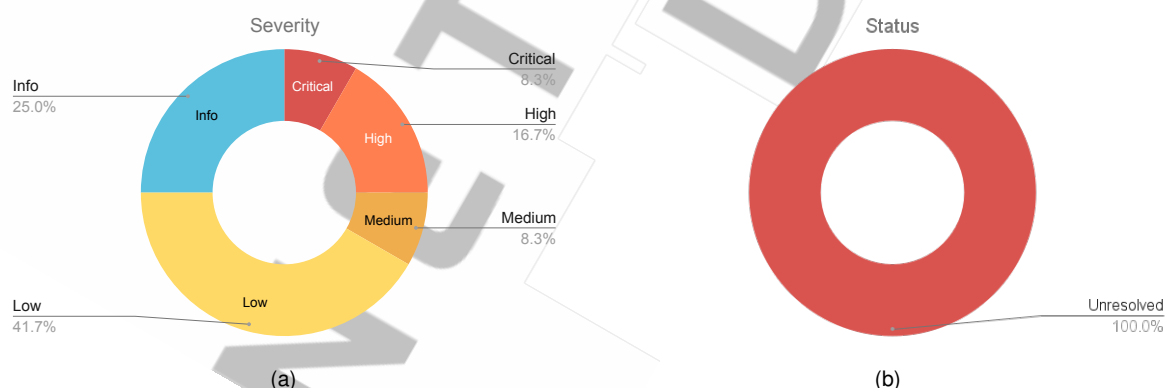


Fig. 1: Distribution of issues: Critical (1), High (2), Medium (1), Low (5), Undetermined (0), Informational (3), Best Practices (0). Distribution of status: Fixed (0), Acknowledged (0), Mitigated (0), Unresolved (12)

Summary of the Audit

Audit Type	Security Review
Initial Report	March 2, 2026
Final Report	-
Initial Commit	73fc2360f936020901a2c57c29fb71c1d3182888
Final Commit	-
Documentation Assessment	Low
Test Suite Assessment	Low

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	WhitelistRegistry.sol	107	11	10.3%	27	145
2	TieredLiquidationMorpho.sol	374	19	5.1%	72	465
3	libraries/HealthFactorLib.sol	29	6	20.7%	6	41
	Total	510	36	7.1%	105	651

3 Summary of Issues

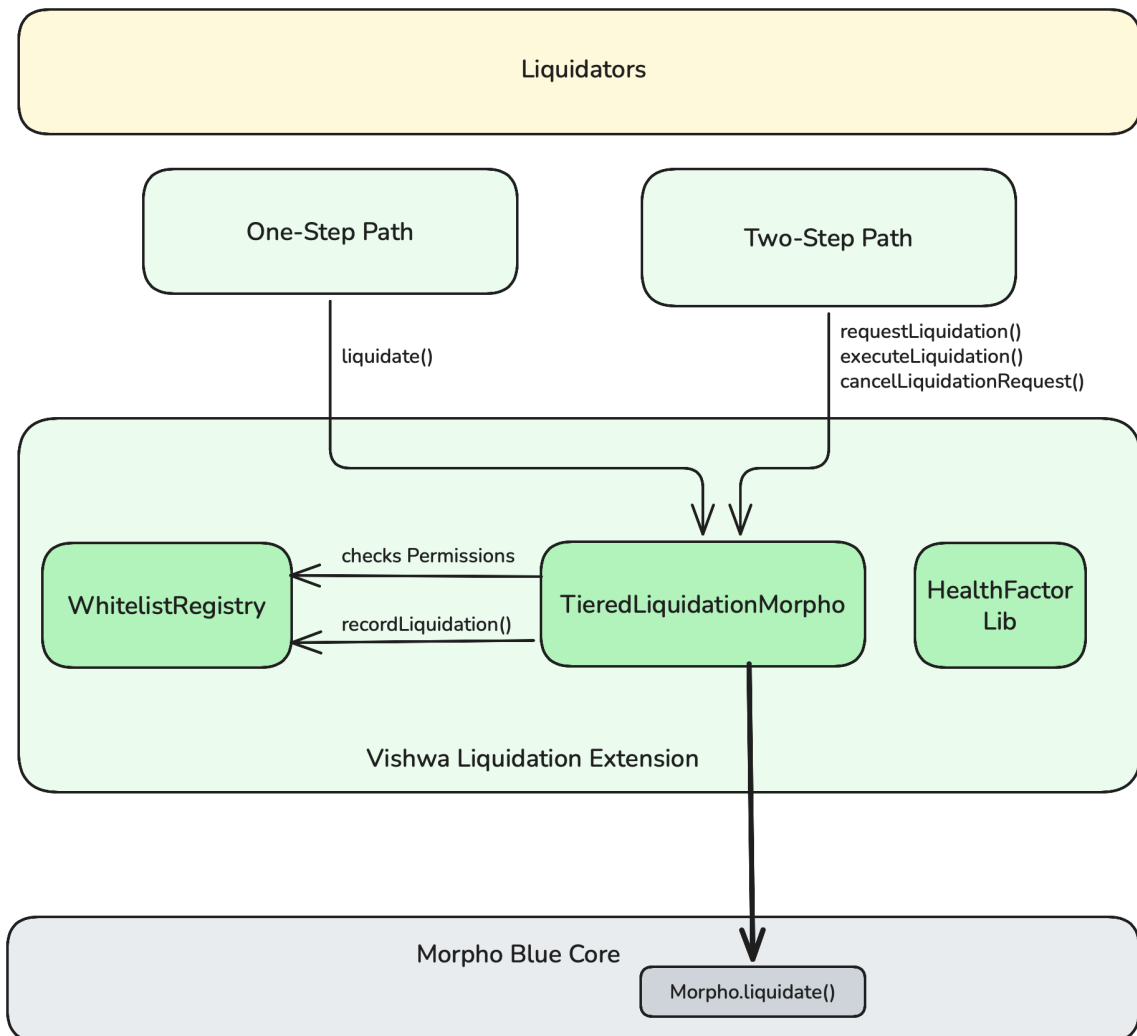
	Finding	Severity	Update
1	The TieredLiquidationMorpho extension can be bypassed through direct calls to the core Morpho contract	Critical	Unresolved
2	Inconsistent liquidation incentive factor causes reverts and incorrect fee calculations	High	Unresolved
3	Health factor check in the TieredLiquidationMorpho contract uses stale market data	High	Unresolved
4	Reentrancy in _enforceNoActiveLock allows cooldown bypass and excess liquidation	Medium	Unresolved
5	Pending liquidations can be executed in disabled markets	Low	Unresolved
6	Residual allowance in _executeMorphoLiquidation prevents subsequent liquidations for non-standard tokens	Low	Unresolved
7	Unauthorized liquidations possible during market initialization	Low	Unresolved
8	Whitelisted addresses cannot be restricted from performing one-step liquidations	Low	Unresolved
9	lockDuration changes affect pending liquidation requests	Low	Unresolved
10	Inconsistent expiry boundary handling across liquidation request lifecycle	Info	Unresolved
11	The TieredLiquidationMorpho contract does not implement IMorphoLiquidateCallback	Info	Unresolved
12	claimFailedRefund permanently locks ETH for contracts that cannot receive it	Info	Unresolved

4 System Overview

The Vishwa protocol introduced a **Tiered Liquidation** system that extends Morpho's native liquidation logic with configurable access controls, incentive structures, and commitment mechanisms.

4.1 Architecture & Core Components

The system employs a layered architecture in which extension contracts act as intermediaries between liquidators and the Morpho Blue singleton. It consists of three main contracts:



4.1.1 TieredLiquidationMorpho

The central entry point for the tiered system. It wraps Morpho Blue's liquidation function and enforces market-specific configurations, including:

- **Cooldowns:** Minimum time required between successive liquidations of the same borrower.
- **Liquidation Limits:** Constraints on the maximum fraction of a position (`maxLiquidationRatio`) and minimum collateral seized (`minSeizedAssets`).
- **Protocol Fees & Bonuses:** Vishwa calculates incentives independently of Morpho Blue's native formula. The protocol collects a fee from the **liquidator's bonus**:

- Total Bonus = $\frac{\text{Seized Assets} \times \text{Liquidation Bonus}}{1 + \text{Liquidation Bonus}}$
- Protocol Fee = Total Bonus $\times$ Fee Percentage

4.1.2 WhitelistRegistry

A per-market access control registry. It manages "Market Admins" who can whitelist specific addresses as authorized liquidators. It tracks liquidation counts and determines whether a market is in **Public** or **Whitelist-only** mode.

It provides the `canLiquidate(marketId, liquidator)` function which determines authorization:

- If whitelist mode is **disabled** for a market, all addresses are authorized (returns true).
- If whitelist mode is **enabled**, only the market admin, the registry owner, or explicitly whitelisted addresses may liquidate.

Markets must be initialized by the owner. Once initialized, the market admin manages the liquidator set. The registry enforces that at least one liquidator remains when whitelist mode is active.

4.1.3 HealthFactorLib

A utility library used to calculate the health factor $((\text{collateral} \times \text{price} \times \text{LLTV}) / \text{borrowed})$ and determine the precise seizable and repayable limits based on the tiered configuration.

4.2 Liquidation Flows

The protocol provides two distinct operational flows for liquidating unhealthy positions.

4.2.1 One-Step Liquidation (`liquidate`)

This method allows any user (or a whitelisted liquidator) to immediately liquidate an unhealthy position, provided the market is configured for public liquidations. The liquidator repays a portion of the borrower's debt and, in return, receives a corresponding amount of the borrower's collateral at a discount. This process is governed by market-specific parameters such as cooldown periods between liquidations and maximum liquidation ratios.

4.2.2 Two-Step Liquidation

A commitment-based flow exclusive to whitelisted liquidators, designed to prevent front-running and provide execution certainty.

- **Step 1: Request (`requestLiquidation`):** The liquidator posts an ETH deposit to "lock" a borrower's position. For the `lockDuration`, no other liquidator (including one-step users) can target this position.
- **Step 2: Execute (`executeLiquidation`):** The original requester completes the liquidation within the lock window. The ETH deposit is refunded upon success.
- **Cancellation:** If the lock expires without execution, the request can be cancelled by anyone to release the position, and the deposit is returned to the requester.

Failed ETH refunds are tracked in `failedRefunds` and can be claimed later via `claimFailedRefund()`.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- High:** The issue is trivial to exploit and has no specific conditions that need to be met;
- Medium:** The issue is moderately complex and may have some conditions that need to be met;
- Low:** The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- High:** The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- Medium:** The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- Low:** The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Critical] The TieredLiquidationMorpho extension can be bypassed through direct calls to the core Morpho contract

File(s): `src/extensions/TieredLiquidationMorpho.sol`, `src/Morpho.sol`

Description: The TieredLiquidationMorpho contract is an external wrapper that introduces additional safeguards on top of Morpho's base liquidation mechanism. These safeguards include a whitelist registry restricting who can perform liquidations, a `maxLiquidationRatio` cap limiting how much collateral can be seized in a single liquidation, cooldown periods between consecutive liquidations of the same borrower, protocol fees on liquidation bonuses, and a two-step liquidation flow that gives whitelisted liquidators an exclusive lock period backed by an ETH deposit. Internally, the extension delegates the actual liquidation to the core Morpho contract:

```

1  function _executeMorphoLiquidation(
2      ...
3  ) internal
4      returns (uint256 actualSeized, uint256 actualRepaid)
5  {
6      // ...
7      (actualSeized, actualRepaid) = MORPHO.liquidate(
8          marketParams, borrower,
9          seizedAssets, repaidShares, data
10     );
11     // ...
12 }

```

However, the core Morpho contract's `liquidate(...)` function is fully permissionless and directly callable by any user:

```

1  function liquidate(
2      MarketParams memory marketParams,
3      address borrower,
4      uint256 seizedAssets,
5      uint256 repaidShares,
6      bytes calldata data
7      // @audit-issue No access control is present.
8  ) external returns (uint256, uint256) {
9      // ...
10 }

```

Since TieredLiquidationMorpho does not restrict direct access to the underlying Morpho singleton, any user can bypass all protective mechanisms by liquidating positions directly through the base contract. Specifically:

- Non-whitelisted actors can liquidate directly, rendering the WhitelistRegistry ineffective;
- The `maxLiquidationRatio` cap is unenforceable since Morpho permits seizing up to 100% of collateral;
- Cooldown periods between liquidations are ignored;
- Protocol fees on liquidation bonuses are never collected;
- The two-step liquidation flow is vulnerable to front-running: any user can monitor `LiquidationRequested` events and front-run the whitelisted liquidator's `executeLiquidation(...)` call by liquidating the position directly on Morpho, causing the whitelisted liquidator's transaction to revert with `HealthyPosition`;

Recommendation(s): Consider introducing a mechanism at the Morpho core level to enforce that liquidations for markets can only be performed through the designated extension contract.

Status: Unresolved

Update from the client:

6.2 [High] Inconsistent liquidation incentive factor causes reverts and incorrect fee calculations

File(s): `src/extensions/TieredLiquidationMorpho.sol`, `src/Morpho.sol`

Description: The TieredLiquidationMorpho contract computes the liquidation incentive factor (LIF) as a flat admin-configured value stored in `config.liquidationBonus`:

```
1 pd.liquidationIncentiveFactor =
2   WAD + config.liquidationBonus;
```

In contrast, the core Morpho contract derives the LIF dynamically from the market's LLTV using the following formula:

```
1 uint256 liquidationIncentiveFactor = UtilsLib.min(
2   MAX_LIQUIDATION_INCENTIVE_FACTOR,
3   WAD.wDivDown(
4     WAD - LIQUIDATION_CURSOR.wMulDown(
5       WAD - marketParams.lltv
6     )
7   )
8 );
```

Since Morpho always applies its own formula regardless of what TieredLiquidationMorpho computed, the configurable bonus only governs how many loan tokens are pre-pulled from the liquidator via `estimatedRepay`, how much collateral is requested in the two-step `executeLiquidation(...)` path, and how the protocol fee is calculated in `_deductProtocolFee(...)`.

Any divergence between the two values can lead to the following issues: First, the extension pre-pulls loan tokens from the liquidator based on its own LIF calculation. If the extension's LIF is lower than Morpho's actual LIF, the pre-pulled amount (even with the 20% buffer applied via $\ast 12 / 10$) may not cover the actual repayment Morpho requires, causing the liquidation transaction to revert.

Second, the `_deductProtocolFee(...)` function computes fees based on `config.liquidationBonus`, not the actual bonus that Morpho applied. This means protocol fees may be overcharged or undercharged relative to the true liquidation bonus:

```
1 function _deductProtocolFee(...) internal
2   returns (uint256)
3 {
4   // ...
5   // @audit-issue Uses the configured liquidationBonus
6   uint256 lif = WAD + config.liquidationBonus;
7   uint256 totalBonus =
8     seizedAssets.mulDivDown(
9       config.liquidationBonus, lif
10    );
11   uint256 feeAmount =
12     totalBonus.mulDivDown(config.protocolFee, WAD);
13   // ...
14 }
```

Third, the `executeLiquidation(...)` function in the two-step flow computes `totalSeized` using the extension's LIF, but Morpho may compute a different amount based on its own formula, leading to unexpected collateral seizure outcomes:

```
1 function executeLiquidation(...) external
2   returns (...)
3 {
4   // ...
5   // @audit-issue Uses extension's LIF, not Morpho's.
6   uint256 lif = WAD + config.liquidationBonus;
7   uint256 debtToRepay =
8     borrowed.mulDivDown(storedRatio, WAD);
9   uint256 totalSeized = debtToRepay
10     .mulDivUp(lif, WAD)
11     .mulDivUp(ORACLE_PRICE_SCALE, collateralPrice);
12   // ...
13 }
```

Recommendation(s): Consider replicating Morpho's liquidation incentive factor formula on-chain within the TieredLiquidationMorpho contract rather than relying on manual admin configuration.

Status: Unresolved

Update from the client:

6.3 [High] Health factor check in the TieredLiquidationMorpho contract uses stale market data

File(s): `src/extensions/TieredLiquidationMorpho.sol`

Description: The TieredLiquidationMorpho contract wraps Morpho Blue's liquidation functionality. Before allowing a liquidation to proceed, it validates the borrower's health factor by calling the `_loadAndValidatePosition(...)` internal function. This function reads the market state via `MORPHO.market(...)` and position data via `MORPHO.position(...)`, both of which are view functions that return raw storage values without accruing pending interest.

```

1  function _loadAndValidatePosition(
2      ...
3  ) internal view returns (PositionData memory pd) {
4      // ...
5      // @audit-issue The borrowed amount is computed from
6      // stale totalBorrowAssets that have not accrued interest.
7      pd.borrowed = uint256(pos.borrowShares).toAssetsUp(
8          m.totalBorrowAssets, m.totalBorrowShares
9      );
10     pd.healthFactor =
11         HealthFactorLib.calculateHealthFactor(
12             pos.collateral,
13             pd.collateralPrice,
14             pd.borrowed,
15             marketParams.lltv
16         );
17     // @audit-issue A position that is unhealthy after
18     // interest accrual may appear healthy here.
19     if (pd.healthFactor >= WAD) revert HealthyPosition();
20     // ...
21 }

```

In Morpho Blue, interest accrues only during state-changing calls, such as `liquidate(...)`. Since interest only increases the total borrow amount, the stale `totalBorrowAssets` value underestimates the actual debt. This means positions that have become unhealthy after interest accrual may still appear healthy to the extension's check, causing the `HealthyPosition()` revert to trigger and block legitimate liquidations.

Additionally, the `calculateLiquidationLimits(...)` function uses the same stale borrowed value, meaning the maximum seizable collateral and repayable debt are understated. This can delay or completely block liquidations for positions that are just past the unhealthy threshold. In volatile markets or markets with long periods of inactivity, this could contribute to bad debt accumulation.

Recommendation(s): Consider using the up-to-date Morpho state when calculating borrowed, health factor, and liquidation limits. One way of fetching market data after interest accrual is using Morpho's `MorphoBalancesLib` library.

Status: Unresolved

Update from the client:

6.4 [Medium] Reentrancy in _enforceNoActiveLock allows cooldown bypass and excess liquidation

File(s): [src/extensions/TieredLiquidationMorpho.sol](#)

Description: The _enforceNoActiveLock function uses an unsafe push pattern to refund ETH deposits to the previous liquidator. When an expired request is found, it deletes the request and then sends ETH via _safeTransferETH, which performs a raw low-level. This hands execution control to the recipient, who can re-enter any function in the contract.

```

1 function _enforceNoActiveLock(...) internal {
2     LiquidationRequest storage request = liquidationRequests[marketId][borrower];
3     if (request.status == LiquidationStatus.Pending) {
4         // ...
5         delete liquidationRequests[marketId][borrower];
6         // @audit low-level call to the request initiator
7         if (deposit > 0) _safeTransferETH(orig, deposit);
8         emit LiquidationRequestCancelled(marketId, borrower, msg.sender, true);
9     }

```

This function is called at the beginning of both liquidate and requestLiquidation. A whitelisted liquidator operating through a smart contract can exploit this to bypass the cooldown and the maxLiquidationRatio cap. Consider the following scenario:

1. The attacker calls requestLiquidation for a target borrower and let the request expire;
2. Another liquidator (or the attacker) calls liquidate for the same borrower. Inside _enforceNoActiveLock, the expired request is deleted, and the attacker's deposit is refunded via the ETH callback;
3. In their receive() function, the attacker re-enters requestLiquidation for the same borrower, recycling the just-refunded ETH as the new deposit. A new LiquidationRequest is stored with the attacker as the liquidator;
4. The outer liquidate resumes: validates the position (still unhealthy, no cooldown), executes the Morpho liquidation seizing up to maxLiquidationRatio of collateral, and sets lastLiquidationTime[marketId][borrower] = block.timestamp;
5. The attacker calls executeLiquidation using the request created in step 3. If the position remains unhealthy, a second Morpho liquidation executes.

The result is that the borrower suffers two successive liquidations within a single cooldown window. If maxLiquidationRatio is 50%, the attacker seizes 50% in the first liquidation and then up to 50% of the remaining position in the second, achieving a total collateral seizure that is higher than the protocol intended cap at 50% per cooldown period.

Recommendation(s): Consider replacing the push-based ETH refund in _enforceNoActiveLock with a pull-only pattern. Alternatively, consider adding a reentrancy guard to all external state-changing functions in the contract.

Status: Unresolved

Update from the client:

6.5 [Low] Pending liquidations can be executed in disabled markets

File(s): [src/extensions/TieredLiquidationMorpho.sol](#)

Description: The protocol allows admins to disable specific markets by setting the enabled flag to false. This is intended to act as a kill switch or emergency stop for market activities.

When a market is disabled, the one-step liquidation process correctly reverts, and new two-step liquidation requests are blocked within the requestLiquidation function.

However, the executeLiquidation function does not verify the config.enabled status. This enables liquidators who submitted a requestLiquidation while the market was active to still call executeLiquidation to finalize the process after the market has been disabled by the owner.

This creates an inconsistency in the protocol's emergency controls. If an owner disables a market, the expectation is that all liquidation activities through the extension should halt immediately. Allowing pending two-step liquidations to execute against a disabled market undermines the owner's ability to use the enabled flag as an emergency stop.

Recommendation(s): Consider adding a check in the executeLiquidation function to check that the market is still enabled before proceeding with the liquidation.

Status: Unresolved

Update from the client:

6.6 [Low] Residual allowance in `_executeMorphoLiquidation` prevents subsequent liquidations for non-standard tokens

File(s): `src/extensions/TieredLiquidationMorpho.sol`

Description: The `_executeMorphoLiquidation` function is used to handle the repayment of debt and seizure of collateral during a Morpho liquidation. To facilitate this, the contract pulls `estimatedRepay` tokens from the liquidator and must then authorize the MORPHO contract to spend these tokens. The contract currently uses a raw call to grant `type(uint256).max` allowance to the MORPHO contract on every single invocation of the function.

```
1 function _executeMorphoLiquidation(  
2     MarketParams calldata marketParams, address borrower,  
3     uint256 seizedAssets, uint256 repaidShares, uint256 estimatedRepay, bytes calldata data  
4 ) internal returns (uint256 actualSeized, uint256 actualRepaid) {  
5     address loanToken = marketParams.loanToken;  
6     IERC20(loanToken).safeTransferFrom(msg.sender, address(this), estimatedRepay);  
7     // ...  
8     // @audit-issue Unlimited approval  
9     (bool success,) = loanToken.call(  
10         abi.encodeWithSignature("approve(address,uint256)", address(MORPHO), type(uint256).max)  
11     );  
12     if (!success) revert ApproveFailed();  
13  
14     (actualSeized, actualRepaid) = MORPHO.liquidate(marketParams, borrower, seizedAssets, repaidShares, data);  
15     // ...  
16 }
```

This implementation presents two significant issues. First, it violates the principle of least privilege by granting an unlimited allowance when only a specific `estimatedRepay` amount is required.

Second, the contract does not reset the allowance to zero after the liquidation. Certain popular tokens, most notably USDT, contain a requirement that the current allowance must be zero before a new non-zero allowance can be set.

Because `type(uint256).max` is set in the first call and persists as a residual allowance, every subsequent attempt to call `_executeMorphoLiquidation` for that token will revert.

Recommendation(s): Consider using OpenZeppelin's `forceApprove` utility to handle tokens with non-standard behaviour. Ideally, the contract should approve only the exact `estimatedRepay` amount and reset it to zero after the liquidation call.

Status: Unresolved

Update from the client:

6.7 [Low] Unauthorized liquidations possible during market initialization

File(s): `src/extensions/WhitelistRegistry.sol`

Description: When a new market is initialized via the `initializeMarket(...)` function in the `WhitelistRegistry` contract, only the market admin is set. The whitelist itself is not enabled in the same transaction:

```

1  function initializeMarket(...) external onlyOwner {
2      // ...
3      // @audit-issue The whitelist is not enabled here.
4      marketAdmin[marketId] = admin;
5      emit MarketAdminTransferred(
6          marketId, address(0), admin
7      );
8  }

```

The `canLiquidate(...)` function returns true for all callers when the whitelist is disabled:

```

1  function canLiquidate(
2      Id marketId, address liquidator
3  ) external view returns (bool) {
4      // @audit-issue All addresses are treated as authorized when the whitelist is disabled.
5      if (!isWhitelistEnabled[marketId]) return true;
6      // ...
7  }

```

This creates a window between market initialization and the subsequent `setWhitelistMode(...)` call during which every address is treated as an authorized liquidator. If the corresponding market is already configured in `TieredLiquidationMorpho`, any user can perform liquidations through the extension during this window, bypassing the intended whitelist restriction.

Recommendation(s): Consider enabling the whitelist by default within `initializeMarket(...)` so that the market starts in a restrictive state immediately upon initialization, eliminating the permissive window between setup transactions.

Status: Unresolved

Update from the client:

6.8 [Low] Whitelisted addresses cannot be restricted from performing one-step liquidations

File(s): `src/extensions/TieredLiquidationMorpho.sol`

Description: The `TieredLiquidationMorpho` contract offers two liquidation modes: a one-step public liquidation and a two-step flow that requires an ETH deposit and grants exclusive lock periods to whitelisted liquidators. The market admin can configure these modes independently using the `twoStepLiquidationEnabled` flag in the `MarketConfig`.

However, the one-step `liquidate(...)` function allows any whitelisted address to perform a liquidation even when `twoStepLiquidationEnabled` is set to true:

```

1  function liquidate(...) external returns (...) {
2      // ...
3      bool isWhitelisted =
4          WHITELIST_REGISTRY.canLiquidate(
5              marketId, msg.sender
6          );
7      // @audit-issue Whitelisted users bypass the public liquidation check entirely.
8      if (
9          !config.publicLiquidationEnabled
10         && !isWhitelisted
11     ) revert PublicLiquidationNotEnabled();
12     // ...
13 }

```

There is no configuration option to restrict whitelisted liquidators to exclusively use the two-step flow. This means even when a market admin intends for all liquidations to go through the two-step process, which requires an ETH deposit and provides exclusive lock periods, whitelisted addresses can simply use the one-step path and skip these requirements entirely.

Recommendation(s): Consider introducing a separate configuration flag that allows the market admin to disable one-step liquidations for whitelisted addresses, ensuring they are required to use the two-step flow when the admin deems it necessary.

Status: Unresolved

Update from the client:

6.9 [Low] lockDuration changes affect pending liquidation requests

File(s): `src/extensions/TieredLiquidationMorpho.sol`

Description: The TieredLiquidationMorpho contract allows the owner to change the lockDuration parameter of a market at any time via the `configureMarket(...)` function. The expiration time of a pending liquidation request is computed dynamically by combining the stored `requestTimestamp` with the current `lockDuration`:

```
1  uint256 expiresAt =
2      uint256(request.requestTimestamp)
3      + config.lockDuration;
```

Since the expiry is not stored at request creation time but computed on-the-fly, any change to `lockDuration` retroactively applies to all pending requests. If the owner increases `lockDuration`, the liquidator's ETH deposit is locked for longer than originally anticipated. If the owner decreases it, previously valid requests may instantly appear expired, causing the `executeLiquidation(...)` function to revert with `LiquidationRequestExpired` and the liquidator to lose their execution window.

Recommendation(s): Consider storing the `lockDuration` or the computed `expiresAt` value per-request at creation time rather than reading it dynamically from the current configuration.

Status: Unresolved

Update from the client:

6.10 [Info] Inconsistent expiry boundary handling across liquidation request lifecycle

File(s): `src/extensions/TieredLiquidationMorpho.sol`

Description: The protocol manages two-step liquidation requests that are valid until a specific `expiresAt` timestamp. Three different functions evaluate whether a request has expired, but they use inconsistent comparison operators. This creates a state where, at the exact timestamp of expiry (`block.timestamp == expiresAt`), the request is simultaneously considered "active" by some functions and "expired" by others.

The `executeLiquidation` function treats the request as valid if the current time is not strictly greater than the expiry:

```
1  function executeLiquidation(...) external returns (uint256 actualSeizedAssets, uint256 actualRepaidAssets) {
2      // ...
3      uint256 expiresAt = uint256(request.requestTimestamp) + config.lockDuration;
4      if (block.timestamp > expiresAt) revert LiquidationRequestExpired();
5      // ...
6  }
```

Similarly, the `cancelLiquidationRequest` function prevents third-party cancellations at the boundary:

```
1  function cancelLiquidationRequest(...) external {
2      // ...
3      uint256 expiresAt = uint256(request.requestTimestamp) + marketConfigs[marketId].lockDuration;
4      bool isExpired = block.timestamp > expiresAt;
5      // ...
6  }
```

However, the internal `_enforceNoActiveLock` function, which is called during new liquidation attempts to clear old locks, treats the boundary as expired:

```
1  function _enforceNoActiveLock(...) internal {
2      LiquidationRequest storage request = liquidationRequests[marketId][borrower];
3      if (request.status == LiquidationStatus.Pending) {
4          uint256 expiresAt = uint256(request.requestTimestamp) + lockDuration;
5          if (block.timestamp < expiresAt) revert LiquidationRequestLocked();
6          // ...
7      }
8  }
```

Consequently, at the exact second of expiry, while a liquidator request can still be executed through `executeLiquidation`, any other liquidator calling a one-step liquidation will trigger `_enforceNoActiveLock`, which will silently cancel the pending request.

Recommendation(s): Consider adopting a consistent boundary comparison across the entire codebase.

Status: Unresolved

Update from the client:

6.11 [Info] The TieredLiquidationMorpho contract does not implement IMorphoLiquidateCallback

File(s): [src/extensions/TieredLiquidationMorpho.sol](#)

Description: The `liquidate(...)` and `executeLiquidation(...)` functions in the `TieredLiquidationMorpho` contract accept a `data` parameter that is forwarded to the upstream `MORPHO.liquidate(...)` call. Inside Morpho Blue, when the `data` parameter has a length greater than 0, the following callback is triggered:

```
1  if (data.length > 0)
2      IMorphoLiquidateCallback(msg.sender)
3      .onMorphoLiquidate(repaidAssets, data);
```

In this context, `msg.sender` is the `TieredLiquidationMorpho` contract. However, this contract does not implement the `IMorphoLiquidateCallback` interface or the `onMorphoLiquidate(...)` function, meaning any call with non-empty data will revert.

This breaks flash-liquidation patterns where the callback is used to source repayment funds, and any liquidator passing non-empty data will lose gas to a guaranteed revert.

Recommendation(s): Consider either implementing the `onMorphoLiquidate(...)` callback in the `TieredLiquidationMorpho` contract to support flash liquidation strategies, or removing the `data` parameter entirely and always passing empty bytes to `Morpho.liquidate()`.

Status: Unresolved

Update from the client:

6.12 [Info] `claimFailedRefund` permanently locks ETH for contracts that cannot receive it

File(s): [src/extensions/TieredLiquidationMorpho.sol](#)

Description: The contract implements a fallback mechanism for failed ETH transfers. When a liquidation occurs, if the ETH transfer to the recipient fails, the `_safeTransferETH` helper function catches the failure and records the amount in a `failedRefunds` mapping instead of reverting the entire transaction.

However, the recovery function `claimFailedRefund` attempts to send the recorded ETH back to `msg.sender` using a low-level call, but it does not allow the caller to specify a different recipient address.

```
1  function claimFailedRefund() external {
2      uint256 amount = failedRefunds[msg.sender];
3      if (amount == 0) revert NoFailedRefund();
4      failedRefunds[msg.sender] = 0;
5      //@audit sends funds to the same address
6      (bool success, ) = payable(msg.sender).call{value: amount}("");
7      if (!success) revert RefundClaimFailed();
8      emit RefundClaimed(msg.sender, amount);
9  }
```

If the original transfer failed because the liquidator is a smart contract that cannot accept ETH, the `claimFailedRefund` function will fail for the exact same reason. Since there is no way to redirect the payment to a different address, the ETH becomes permanently locked within the `TieredLiquidationMorpho` contract.

Recommendation(s): Consider modifying the `claimFailedRefund` function to accept a recipient address parameter. This allows users to redirect their trapped refunds to an address that can successfully receive them.

Status: Unresolved

Update from the client:

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Vishwa Morpho Blue Protocol's contracts documentation

The Vishwa team provided an overview of the system components during the kick-off call with an explanation of the intended functionalities and use cases of the system. Moreover, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

Remarks about the Vishwa's Morpho Blue Protocol's test suite

The current test suite for the Vishwa contracts is non-functional. Some tests attempt to invoke functions that do not exist in the current codebase, indicating that the tests are outdated or misaligned with the implemented contracts. As a result, the protocol effectively lacks a reliable and formal test suite for these extensions. We strongly recommend implementing and maintaining a scenario-driven testing framework that accurately reflects the deployed code prior to production.

8.1 Automated Tools

8.1.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at <https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.